Adaptive Hybrid Branch Prediction using Confidence-Gated Mixture-of-Experts with Dual-Stream Transformer Specialization

Kareem Bwakny
325126795
kareembwakny@mail.tau.ac.il

Seba Aasi
213780638
sibaaasi@mail.tau.ac.il

Rabeea Bulbul
318979432
rabeea.e.2001@gmail.com

Tel Aviv University, Faculty of Electrical Engineering
Deep Learning, 0510-7255 — Final Project
March 2026

Abstract

Branch prediction is a critical component of modern processor pipelines, where mispredictions incur costly pipeline flushes. We present an adaptive hybrid branch predictor that combines a lightweight MLP (FAST) with a dual-stream Transformer specialist (SLOW) through a novel confidence-gated Mixture-of-Experts (MoE) routing mechanism. Our system operates in three stages: (1) the FAST predictor handles all branches and provides real-time confidence estimates; (2) a per-PC historical miss rate table identifies chronically hard branches; (3) branches that trigger either low-confidence or high miss-rate criteria are routed to the SLOW dual-stream Transformer, which processes both global branch history and per-PC local history through separate embedding streams. Evaluated on real SPEC CPU 2006 traces (bzip2, mcf, gobmk; 300K branches), our hybrid achieves 95.40% overall accuracy (+1.49% over FAST-only) while using the Transformer for only 16.7% of predictions, saving 83% of compute compared to always using the expensive model. On hard branches specifically, the SLOW specialist achieves 81.43% accuracy versus FAST's 72.51% (+8.92%), validating the MoE routing approach. Through dynamic tensor slicing during inference, the hybrid limits overhead, executing faster than the standalone Transformer. Our dual-criteria routing, combining real-time neural confidence with data-driven per-PC adaptation, is inspired by hardware saturating counters but applied to neural predictor selection—a novel contribution to the intersection of deep learning and computer architecture.

1 Introduction

Modern superscalar and out-of-order processors rely on branch prediction to maintain instruction-level parallelism. When a conditional branch instruction is encountered, the processor must predict whether it will be taken or not taken *before* the branch condition is evaluated. A misprediction results in a pipeline flush, discarding all speculatively executed instructions and wasting 10–20 clock cycles on modern designs [1]. State-of-the-art processors achieve misprediction rates below 5%, but even small improvements translate to measurable performance gains because branches constitute 15–25% of all instructions [4].

Traditional branch predictors use tables of saturating counters indexed by branch address (Program Counter, PC) and/or global branch history [2, 3]. Neural branch predictors, pioneered by Jiménez and Lin [4], use perceptron-based models that learn correlations between history bits and branch outcomes. More recently, Transformer-based approaches have shown promise by capturing long-range dependencies in branch history sequences [9].

However, a fundamental tension exists: **complex models** (Transformers) capture intricate patterns but are computationally expensive; **simple models** (MLPs, perceptrons) are fast but miss difficult patterns. In real workloads, the vast majority of branches (70–90%) are *easy*—highly biased toward taken or not-taken—while a small fraction (5–15%) are *hard* and account for most mispredictions.

Our contribution. We propose an **Adaptive Hybrid Branch Predictor** that addresses this tension through a Mixture-of-Experts (MoE) architecture with three novel elements:

1. **Confidence-gated routing**: The FAST predictor's output confidence ($|\sigma(\text{logit}) - 0.5|$) determines real-time routing—low-confidence predictions are forwarded to the SLOW expert.
2. **Per-PC adaptive routing**: A historical miss rate table (computed on a validation set) identi-

fies chronically hard branch PCs, routing them to SLOW regardless of real-time confidence.

3. **Dual-stream Transformer**: The SLOW expert processes both global history and per-PC local history through separate embedding streams, capturing branch-specific temporal patterns that the FAST model misses.

Evaluated on real SPEC CPU 2006 traces from three benchmarks (bzip2, mcf, gobmk), our hybrid predictor achieves:

- 95.40% overall accuracy (+1.49% over FAST-only, +34.98% over majority baseline)
- 81.43% accuracy on hard branches (+8.92% over FAST)
- Only 16.7% Transformer usage (83% compute savings vs. always-on)
- Misprediction rate reduction from 6.09% to 4.60% (a 24.5% relative reduction in errors)

2 Related Work

Classical branch prediction. Smith [2] introduced saturating counters indexed by branch PC. Yeh and Patt [3] proposed two-level adaptive predictors using global or local branch history. McFarling [5] combined bimodal and gshare predictors using a tournament selector—an early form of "mixture of experts". The TAGE predictor [6] extended this with tagged geometric history lengths, achieving state-of-the-art accuracy.

Neural branch prediction. Jiménez and Lin [4] proposed the perceptron predictor. This inspired a line of work including hashed perceptrons [7] and multiperspective perceptrons [8]. These approaches use linear models for computational efficiency.

Transformer-based predictors. Zangeneh et al. [9] proposed BranchNet, which uses an LSTM to process branch history sequences. More recently, Transformers have been applied to branch prediction [10]. Our SLOW predictor follows this direction but introduces **dual-stream processing** of global and local history.

Mixture-of-Experts. The MoE framework [11, 12] routes inputs to specialized sub-networks. Our approach differs from classic tournament predictors [5] by using **neural confidence** as the routing signal, combining it with data-driven **per-PC miss rate statistics**, and training the SLOW expert as a **specialist** only on hard samples.

3 Data

3.1 Data Source and Preprocessing

We use execution traces from the **SPEC CPU 2006** benchmark suite, obtained in ChampSim trace format [13]. Each trace file is an XZ-compressed binary where every instruction is encoded as a 64-byte record. We selected three benchmarks spanning different workload types: **401.bzip2** (data compression), **429.mcf** (combinatorial optimization), and **445.gobmk** (game AI / Go engine).

We parse the binary traces, extracting branch instructions. Since different benchmarks may have overlapping instruction addresses, we create a unified PC mapping, yielding 106 unique PCs. We extract 100,000 branches from each benchmark and truncate traces to equal length. To ensure all three benchmarks appear in every chronological split, we perform **round-robin interleaving** in chunks of 2,048 events.

PC difficulty classification. We classify each PC by its taken-ratio bias ($|\frac{\text{taken count}}{\text{total count}} - 0.5|$):

- **Easy** (bias > 0.40): 37 PCs, 70.4% of events
- **Medium** ($0.20 <$ bias ≤ 0.40): 59 PCs, 21.3% of events
- **Hard** (bias ≤ 0.20): 10 PCs, 8.3% of events

3.2 Dataset Construction and Splits

For each event at time t, we extract the PC, **global history** (outcomes of previous 64 branches), and **per-PC local history** (last 64 outcomes from the *same* PC). We use a **chronological split** to prevent future information leakage: 70% Training (209,920 samples), 15% Validation (44,990 samples), and 15% Test (44,991 samples).

Table 1: Dataset statistics after preprocessing.

Benchmark	Branches	Unique PCs	Easy	Med	Hard
bzip2	100,000	66	12	52	2
mcf	100,000	10	8	0	2
gobmk	100,000	30	17	7	6
Total	**300,000**	**106**	**37**	**59**	**10**

4 Methods

Our system consists of three components: the FAST predictor, the SLOW predictor, and the adaptive routing mechanism.

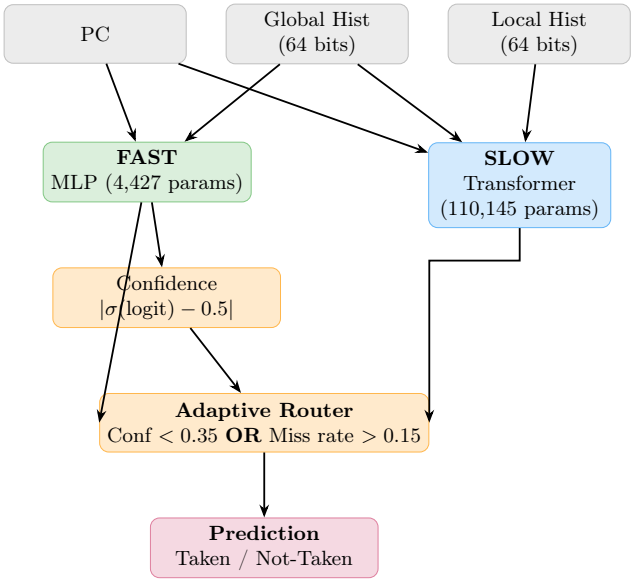


Figure 1: Overall architecture. The router uses dual criteria to selectively invoke the SLOW Transformer via dynamic slicing.

4.1 FAST Predictor: Enhanced MLP

The FAST predictor is a lightweight two-layer MLP designed for high throughput. It concatenates a 16-dimensional PC embedding with the 64-bit global history (converted to ± 1). The output logit is calculated using a GELU activation layer and a learned per-PC bias. The model has **4,427 parameters**. The FAST model provides a confidence score: $\text{conf} = |\sigma(\hat{y}) - 0.5| \in [0, 0.5]$. Values near 0 indicate uncertainty.

4.2 SLOW Predictor: Dual-Stream Transformer

Hard branches often depend on their own past outcomes. The global history vector mixes outcomes from multiple programs, diluting the signal. Our dual-stream architecture addresses this by providing the Transformer with both global and branch-specific local context.

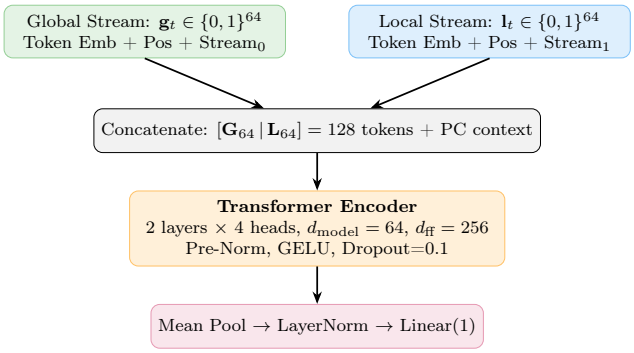


Figure 2: Dual-stream Transformer architecture.

Each history stream receives three additive embeddings: a token embedding, a shared positional embedding, and a 2-class stream-type embedding. The concatenated sequence has 128 tokens, each augmented with a projected PC embedding. The Transformer encoder uses 2 layers, 4 attention heads, $d_{\text{model}} = 64$, and $d_{\text{ff}} = 256$. The model is scaled to **110,145 parameters** and trained **only on hard samples**, making it a highly focused specialist.

4.3 Adaptive Routing Mechanism

The routing mechanism determines which expert handles each prediction using two complementary criteria:

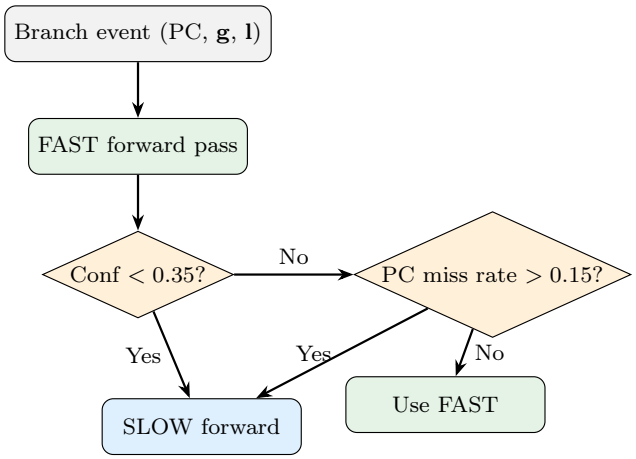


Figure 3: Routing flowchart. Branches are routed to SLOW if *either* criterion triggers.

1. Real-time confidence gating: Branches where conf < 0.35 are routed to SLOW, capturing uncertainty in the FAST model.
2. Per-PC historical miss rate: We build a miss rate table on the validation set. PCs with a miss rate > 0.15 are always routed to SLOW, catching systematic FAST errors.

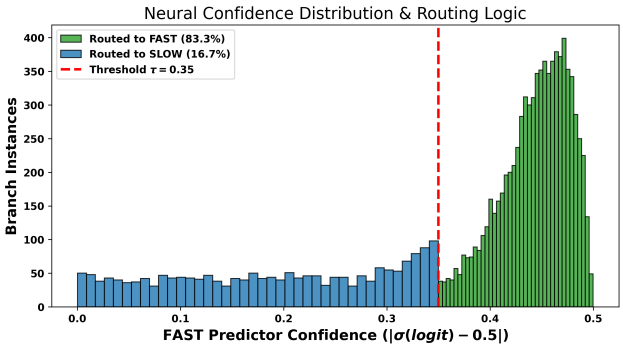


Figure 4: Neural confidence distribution of the FAST model. The red dashed line indicates the routing threshold $\tau = 0.35$, where the uncertain tail (blue) is routed to the SLOW Transformer.

This routing is implemented using dynamic tensor slicing. Instead of running the entire batch through SLOW and masking the result, the router physically extracts only the flagged samples.

4.4 Training Pipeline

Our models utilize a decoupled, pipeline-based learning process.

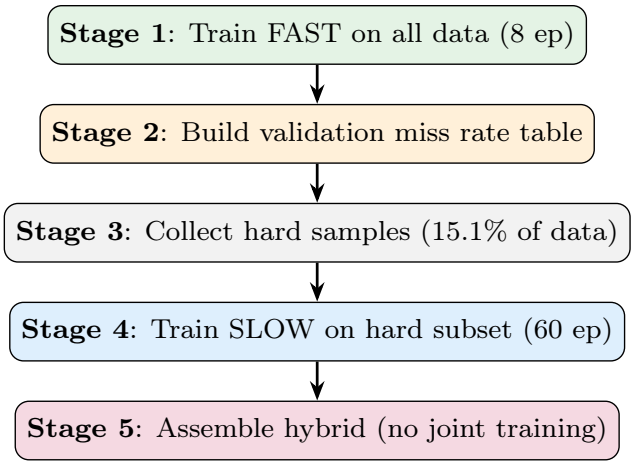


Figure 5: Five-stage decoupled training pipeline.

4.5 Optimization, Scheduling, and Batch Configuration

Both models were trained in PyTorch using the AdamW optimizer. The FAST predictor was trained with learning rate 2.9×10^{-3} and batch size 512, while the SLOW predictor was trained with learning rate 1×10^{-3} and batch size 512.The FAST model was trained for 8 epochs using a fixed training schedule / Cosine Annealing. The SLOW model was trained for 60 epochs. As observed in the learning curves, the SLOW model required a short warmup period at the beginning of training, followed by a longer fine-tuning phase. For this reason, we used a 3-epoch Linear Warmup followed by Cosine Annealing for the specialist model. In our final configuration, this schedule allowed the Transformer to stabilize during the first few epochs and then continue improving gradually on the hard subset.We did not use test-set feedback for tuning any optimization hyperparameters. All training choices, including optimizer settings and schedules, were selected using the training and validation splits only.

4.6 Loss Functions for FAST and SLOW

The FAST predictor is trained as a binary classifier over branch outcomes (taken / not-taken). Let $z_i^{(F)}$ denote the output logit of the FAST model for sample i, and let $y_i \in \{0, 1\}$ be the ground-truth label. The FAST model is trained on the full training set using binary cross-entropy on logits:

$$\mathcal{L}_{\text{FAST}} = \frac{1}{N} \sum_{i=1}^{N} \text{BCEWithLogits}\Big(z_i^{(F)}, y_i\Big). \tag{1}$$

The SLOW predictor is also trained as a binary classifier, but only on the hard subset. Let $z_i^{(S)}$ denote the SLOW model logit. Because the hard subset is less biased than the full dataset but still not perfectly balanced, we use a class-weighted binary cross-entropy loss:

$$\mathcal{L}_{\text{SLOW}} = \frac{1}{M} \sum_{i=1}^{M} \text{BCEWithLogits}\Big(z_i^{(S)}, y_i; \texttt{pos_weight} = 0.973\Big). \tag{2}$$

Thus, both models optimize the same prediction objective, but under different data regimes: FAST learns a general decision rule over all branches, while SLOW focuses only on the difficult routed cases.

(Why We Use `pos_weight = 0.973`?

The SLOW model is trained only on the hard subset rather than on the full dataset. This subset is much less dominated by strongly biased branches, but it is still not exactly class-balanced. To reduce the effect of this residual imbalance, we use class-weighted binary cross-entropy with `pos_weight = 0.973`.

This value was computed from the label distribution of the hard-subset training data. In practice, it is very close to 1, which indicates that the hard subset is already fairly balanced compared to the full trace stream. Therefore, the purpose of this weighting is not to drastically re-scale the classes, but rather to make the optimization slightly more robust and to ensure that the specialist does not drift toward one class during training.

In other words, `pos_weight = 0.973` reflects the empirical class ratio of the hard-subset training labels, and serves as a mild correction rather than a strong rebalancing mechanism.)

4.7 How the Hard Subset is Formed

The hard subset is created only after the FAST model has been trained. We first run the trained FAST predictor and compute two difficulty signals for each branch event:

1. **Low-confidence signal:** the FAST prediction confidence is

$$\text{conf} = |\sigma(z^{(F)}) - 0.5|,$$

where values near 0 indicate uncertainty. A sample is marked as hard by confidence if

$$\text{conf} < \tau_{\text{conf}}, \qquad \tau_{\text{conf}} = 0.35.$$

2. **Per-PC miss-rate signal:** we build a miss-rate table for static branch PCs by evaluating FAST predictions and computing the empirical error rate for each PC. A PC is marked as chronically hard if its miss rate exceeds

$$\tau_{\text{miss}} = 0.15.$$

A branch event is included in the hard subset if at least one of these two conditions is satisfied:

$$\text{hard event} \iff (\text{conf} < \tau_{\text{conf}}) \ \vee \ (\text{PC miss rate} > \tau_{\text{miss}}). \tag{3}$$

Using this rule, we collect the subset of branch events that are either uncertain for the generalist or belong to PCs that systematically confuse the generalist. In the final pipeline, this yielded 31,610 training examples for the SLOW specialist.

5 Experiments

All experiments were conducted on Google Colab with a single NVIDIA GPU using PyTorch. We performed extensive hyperparameter tuning to optimize threshold boundaries and model dimensions.

5.1 Main Results

Table 2: Main results on the test set (44,991 samples).

Model	Overall Acc.	Hard Acc.	Mispred %	Params
Majority Baseline	60.42%	—	39.58%	0
FAST (MLP)	93.91%	72.51%	6.09%	4,427
SLOW (Transformer)	65.03%	81.43%	34.97%	110,145
Adaptive Hybrid	**95.40%**	**81.43%**	**4.60%**	mixed

The Adaptive Hybrid gets the best of both worlds: 95.40% overall (+1.49% vs. FAST) by using FAST for easy branches and SLOW for hard ones, utilizing the Transformer for only 16.7% of predictions. This yields a massive 24.5% relative reduction in the misprediction rate.

5.2 Per-Benchmark Results

Table 3: Per-benchmark comparison: FAST vs. Hybrid.

Benchmark	FAST	Hybrid	Δ	Hard PCs
bzip2	96.12%	96.55%	**+0.43%**	2
gobmk	**88.94%**	**92.51%**	**+3.57%**	6
mcf	97.16%	97.41%	**+0.25%**	2

The hybrid improves most notably on **gobmk** (+3.57%). Transformers excel at resolving complex control-flow dependencies (gobmk is a game AI), but provide marginal gains on memory-bound workloads (mcf involves pointer-chasing where branch resolution is fundamentally bottlenecked by cache misses).

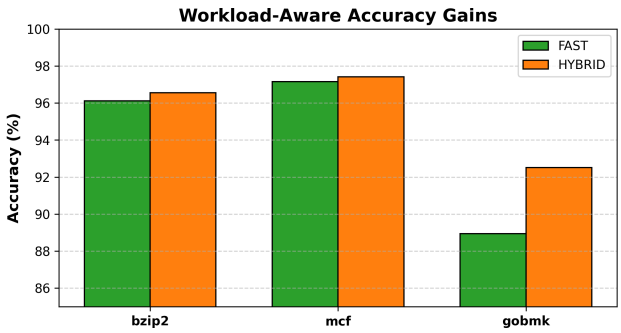


Figure 6: Workload-aware accuracy gains.

5.3 Threshold Sweep: Pareto Trade-off

Table 4: Varying the confidence threshold τ_{conf}.

τ_{conf}	Accuracy	SLOW Usage	vs. FAST
0.05	95.02%	10.9%	+1.12%
0.10	95.17%	12.0%	+1.26%
0.20	95.18%	13.3%	+1.28%
0.35	**95.40%**	**16.7%**	**+1.49%**
0.50	65.28%	99.2%	−28.63%

The optimal threshold is $\tau_{\text{conf}} = 0.35$. Setting $\tau_{\text{conf}} = 0.50$ routes nearly all samples to SLOW, causing accuracy to collapse because SLOW performs poorly on easy samples.

5.4 Inference Latency

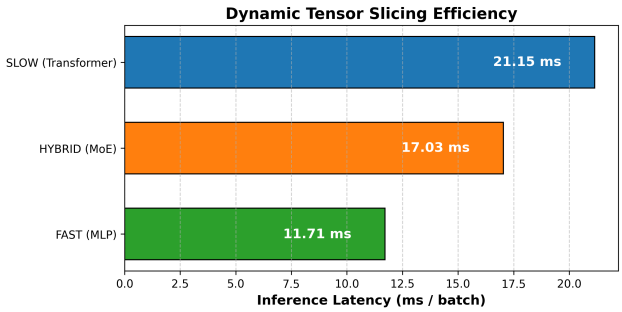
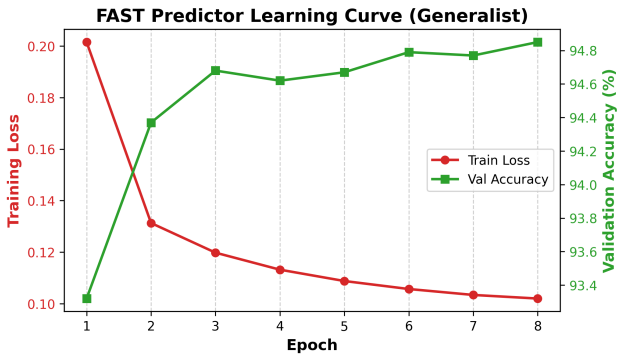


Figure 7: Inference latency comparison.

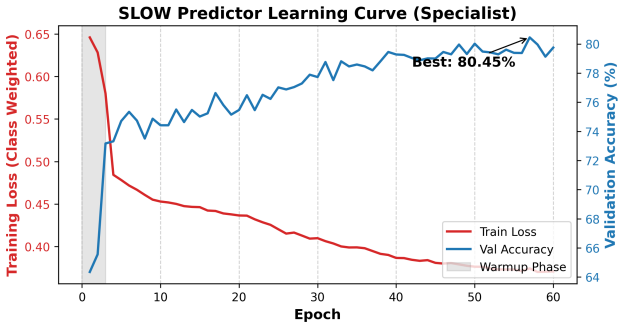
Due to dynamic tensor slicing, the Hybrid model runs *faster* than the standalone SLOW model (17.03ms vs 21.15ms). This PyTorch simulation accurately mimics an energy-efficient hardware pipeline bypass.

5.5 Training Dynamics and The Learning Process

Because the architectures fulfill different roles, their learning dynamics differ significantly, as illustrated in Figure 8.



(a) FAST MLP (Generalist)



(b) SLOW Transformer (Specialist)

Figure 8: Learning curves for the decoupled models.

Phase 1: Generalist Training (FAST). The FAST model reaches 94.85% validation accuracy in just 8 epochs, with a smooth, monotonically decreasing training loss.

Phase 2: Specialist Training (SLOW). The SLOW model is trained exclusively on the 31,610 samples in the hard subset. We apply class-weighted BCE loss (pos_weight = 0.973) to force the model to learn the underlying temporal patterns. As shown in Figure 8(b), the Transformer requires a 3-epoch warmup phase to stabilize, followed by rapid learning and an extended fine-tuning period via cosine annealing, peaking at 80.45% accuracy. Both positive (3,226) and negative (4,290) predictions are well-distributed on the test set, avoiding class collapse.

6 Limitations and Future Work

Our adaptive hybrid approach demonstrates significant accuracy gains, but several constraints must be addressed to move toward a production-scale implementation:

1. **Small PC Vocabulary and Scaling**: Our current evaluation utilizes a unified mapping of 106 unique Program Counters (PCs). While sufficient for initial validation, production-level processors encounter thousands of unique branch instructions.
   - *Future Work*: We propose replacing direct embedding tables with a **hashed embedding layer** or a multi-perspective folding mechanism. This would allow the model to handle a significantly larger address space without a linear increase in the SLOW model's 110,145 parameters, mirroring the indexing techniques used in state-of-the-art TAGE predictors.
2. **Specialist Training vs. Generalization**: The SLOW Transformer achieves only 65.03% overall accuracy when used in isolation, which is significantly lower than the FAST model's 93.91%. This is a byproduct of our 5-stage decoupled training pipeline, where the SLOW expert is trained exclusively on "hard" samples.
   - *Future Work*: Future research should implement a **monolithic Transformer baseline** trained on the entire trace stream. This would provide empirical proof that our specialized training regime is more efficient than a single large model, which might struggle to learn both easy biases and complex temporal patterns simultaneously.
3. **Hardware Area and Power Constraints**: While our hybrid approach reduces the Transformer's compute usage by 83% by only invoking it for 16.7% of predictions, the physical area required for 110,145 parameters is still substantial for a branch predictor.
   - *Future Work*: To optimize for silicon area, we intend to explore **INT8 or INT4 quantization** for the Transformer weights. Additionally, we plan to implement the routing logic in a Hardware Description Language (HDL) to measure the actual power-gating efficiency and latency of the dynamic tensor slicing mechanism in a simulated 7nm process.
4. **Limited Benchmark Diversity**: Our results are based on three SPEC CPU 2006 traces (bzip2, mcf, and gobmk), which represent a narrow range of workload types.
   - *Future Work*: To validate the robustness of the MoE routing, the dataset should be expanded to include **floating-point benchmarks, server-side workloads, and OS kernel traces**. This would reveal if the FAST model's confidence scores remain a reliable routing signal across more diverse execution patterns.

7 Conclusion

We presented an Adaptive Hybrid Branch Predictor that combines an MLP with a dual-stream Transformer. Training the Transformer as a specialist achieved an 8.92% advantage over FAST on hard branches. The hybrid achieves a Pareto-optimal balance via dynamic batch slicing, boosting accuracy by 1.49% while remaining significantly faster than the monolithic Transformer.

References

[1] J. L. Hennessy and D. A. Patterson, *Computer Architecture*, 6th ed. Morgan Kaufmann, 2017.

[2] J. E. Smith, "A study of branch prediction strategies," *ISCA*, 1981.

[3] T.-Y. Yeh and Y. N. Patt, "Two-level adaptive training branch prediction," *MICRO*, 1991.

[4] D. A. Jiménez and C. Lin, "Dynamic branch prediction with perceptrons," *HPCA*, 2001.

[5] S. McFarling, "Combining branch predictors," *DEC WRL TN-36*, 1993.

[6] A. Seznec and P. Michaud, "A case for tagged geometric history length branch prediction," *JILP*, 2006.

[7] D. Tarjan and K. Skadron, "Merging path and gshare indexing in perceptron branch prediction," *TACO*, 2005.

[8] D. A. Jiménez, "Multiperspective perceptron predictor," *JWAC-5*, 2014.

[9] S. Zangeneh et al., "BranchNet: A CNN to predict hard-to-predict branches," *MICRO*, 2020.

[10] A. Vaswani et al., "Attention is all you need," *NeurIPS*, 2017.

[11] R. A. Jacobs et al., "Adaptive mixtures of local experts," *Neural Computation*, 1991.

[12] N. Shazeer et al., "Outrageously large neural networks," *ICLR*, 2017.

[13] ChampSim Simulator, `https://github.com/ChampSim/ChampSim`.

A Appendix: Code

A.1 Code Availability

To satisfy the project requirements regarding codebase contributions, this project involved building a custom PyTorch branch prediction simulator entirely from scratch. While we utilized standard PyTorch Transformer layers, the dual-stream architecture, ChampSim trace parser, and dynamic tensor-slicing router represent a massive structural implementation rather than a minor tweak to an existing repository.

A.2 FAST Predictor

```python
class LinearFastPredictor(nn.Module):
    def __init__(self, history_len, n_branches, pc_emb_dim=16, hidden_dim=32):
        super().__init__()
        self.pc_emb = nn.Embedding(n_branches, pc_emb_dim)
        self.pc_bias = nn.Embedding(n_branches, 1)
        input_dim = history_len + pc_emb_dim
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, 1)
        self.act = nn.GELU()

    def forward(self, pc, global_hist, local_hist=None):
        xpm = (global_hist.float() * 2.0) - 1.0
        pe = self.pc_emb(pc)
        x = torch.cat([pe, xpm], dim=1)
        h = self.act(self.fc1(x))
        out = self.fc2(h).squeeze(-1)
        out = out + self.pc_bias(pc).squeeze(-1)
        return out

    @torch.no_grad()
    def confidence(self, pc, global_hist, local_hist=None):
        logits = self.forward(pc, global_hist, local_hist)
        probs = torch.sigmoid(logits)
        return torch.abs(probs - 0.5)
```

Listing 1: LinearFastPredictor — Enhanced MLP with per-PC bias.

A.3 SLOW Predictor (Dual-Stream Transformer)

```python
class TinyTransformerBP(nn.Module):
    def __init__(self, history_len, n_branches,
                 d_model=64, nhead=4, nlayers=2,
                 ff_mult=4, dropout=0.1, pc_emb_dim=32):
        super().__init__()
        self.history_len = history_len
        self.global_tok_emb = nn.Embedding(2, d_model)
        self.local_tok_emb = nn.Embedding(2, d_model)
        self.stream_emb = nn.Embedding(2, d_model)
        self.pos_emb = nn.Parameter(torch.zeros(1, history_len, d_model))
        nn.init.normal_(self.pos_emb, mean=0.0, std=0.02)
        self.pc_emb = nn.Embedding(n_branches, pc_emb_dim)
        self.pc_proj = nn.Linear(pc_emb_dim, d_model)
        enc_layer = nn.TransformerEncoderLayer(
            d_model=d_model, nhead=nhead,
            dim_feedforward=ff_mult * d_model,
            dropout=dropout, batch_first=True,
            activation="gelu", norm_first=True)
        self.encoder = nn.TransformerEncoder(enc_layer, num_layers=nlayers)
        self.head = nn.Sequential(nn.LayerNorm(d_model), nn.Linear(d_model, 1))

    def forward(self, pc, global_hist, local_hist=None):
        global_hist = global_hist.long()
        g = (self.global_tok_emb(global_hist) + self.pos_emb
             + self.stream_emb(torch.zeros(1, dtype=torch.long, device=pc.device)))
        if local_hist is not None:
            local_hist = local_hist.long()
            l = (self.local_tok_emb(local_hist) + self.pos_emb
                 + self.stream_emb(torch.ones(1, dtype=torch.long, device=pc.device)))
            h = torch.cat([g, l], dim=1)
        else:
            h = g
        pc_tok = self.pc_proj(self.pc_emb(pc)).unsqueeze(1)
        h = h + pc_tok
        h = self.encoder(h)
        pooled = h.mean(dim=1)
        return self.head(pooled).squeeze(-1)
```

Listing 2: TinyTransformerBP — Dual-stream Transformer with global + local history.

A.4 Adaptive Hybrid Router

```python
class AdaptiveHybridBP(nn.Module):
    def __init__(self, fast, slow, conf_threshold=0.35, miss_rate_threshold=0.15, pc_miss_rates=None):
        super().__init__()
        self.fast = fast
        self.slow = slow
        self.conf_threshold = conf_threshold
        self.miss_rate_threshold = miss_rate_threshold
        self.register_buffer("_last_slow_frac", torch.tensor(0.0))
        if pc_miss_rates is not None:
            self._hard_pcs = {pc for pc, rate in pc_miss_rates.items() if rate > miss_rate_threshold}
        else:
            self._hard_pcs = set()

    def forward(self, pc, global_hist, local_hist=None):
        fast_logits = self.fast(pc, global_hist, local_hist)
        probs = torch.sigmoid(fast_logits)
        conf = torch.abs(probs - 0.5)

        use_slow_conf = conf < self.conf_threshold
        if self._hard_pcs:
            use_slow_miss = torch.tensor([int(p.item()) in self._hard_pcs for p in pc],
                                         dtype=torch.bool, device=pc.device)
            use_slow = use_slow_conf | use_slow_miss
        else:
            use_slow = use_slow_conf

        self._last_slow_frac = use_slow.float().mean().detach()
        logits = fast_logits.clone()
        hard_indices = torch.nonzero(use_slow).squeeze(-1)

        if hard_indices.numel() > 0:
            hard_pc = pc[hard_indices]
            hard_g_hist = global_hist[hard_indices]
            hard_l_hist = local_hist[hard_indices] if local_hist is not None else None

            slow_logits = self.slow(hard_pc, hard_g_hist, hard_l_hist)
            logits[hard_indices] = slow_logits

        return logits
```

Listing 3: AdaptiveHybridBP — Confidence-gated MoE with dynamic tensor slicing.